15. *Doubly Linked List –*

MEMORY REFERENCING AND FREEING MEMORY

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;
```

What the above code depicts: –

→ ***struct whose name is DLL is renamed with Node through `typedef` keyword.***

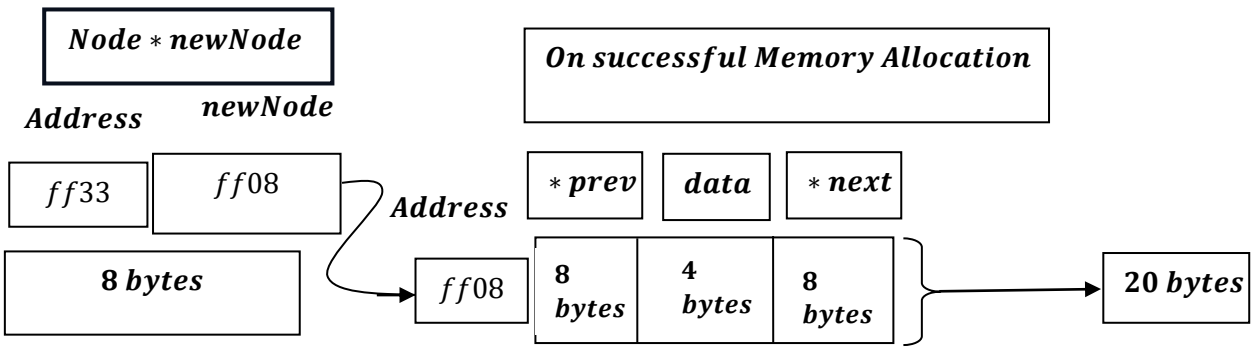
→ ***for struct Node , there will be a memory allocation for integer variable `data`.***

→ ***for struct Node, there will be a memory allocation for pointer next.***

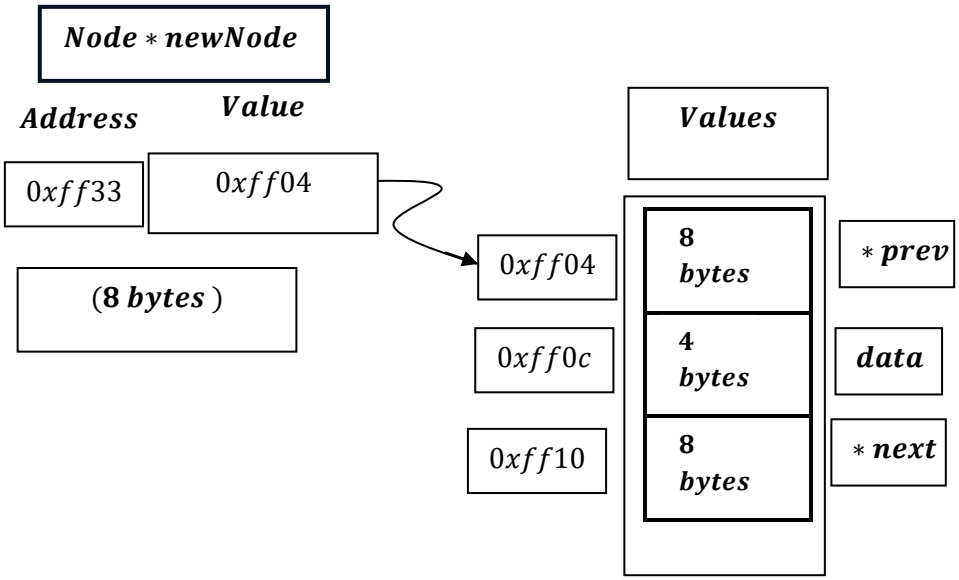
→ ***for struct Node, there will be a memory allocation for pointer prev.***

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

We know [***Based on 64 bit system***]:



But memory allocation doesn't take like this, actually, this is for understanding, and for explanation, memory allocation a little bit more complex:



At now we see newNode pointer , only pointing 0xff04 , but it should update Data part and Next pointer part, how that happens?

→ The compiler knows that the data member is at the very beginning of the struct. Its offset from the start is 0 bytes.

such as: $newNode \rightarrow prev = nullptr$.

$$\rightarrow baseaddress + offset = 0xff04 + 0 = 0xff04.$$

<u>The Compiler's "Secret" Calculation</u>

When the compiler sees newNode->prev =nullptr;, it thinks:

- ***"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)***
- ***"Look up the definition of the Node struct. The prev member is at the beginning, so its offset is 0."***

- *"Calculate the final address: base address + offset => 0xff04 + 0 => 0xff04."*

- *"Write the value `nullptr` or NULL basically (whose output is 0)[pointer points to nothing] to the memory location starting at 0xff04."*

$Next\ is\ Data, an\ integer\ variable$:

$This\ comes\ after\ the\ prev\ pointer\ (8\ bytes).\ Its\ offset\ is\ 8\ bytes.$

$$say, newNode \rightarrow data = 10;$$

$$\rightarrow baseaddress + offset = 0xff04 + 8\ bytes = 0xff0c[12 \rightarrow c\ in\ hex].$$

$$\rightarrow And\ then\ write\ head\ pointer's\ value\ to\ address\ 0xff0c.$$

When it sees newNode->data = integer_value;, it thinks:

1. *"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)*
2. *"Look up the definition of the Node struct. The next member comes after prev pointer, which is 8 bytes long. So, the offset for next is 8."*
3. *"Calculate the final address: base address + offset => 0xff04 + 8 => 0xff0c."*
4. *"Write the integer_value to the memory location starting at 0xff0c."*

$But\ this\ doesn't\ means\ ,value\ of\ newNode\ pointer\ gets\ changed\ and\ it\ starts\ to\ point\ to\ address\ of\ data\ variable, it\ continue\ to\ point\ to\ prev\ pointer$.

$$Next\ is\ next, a\ pointer\ variable:$$

$This\ comes\ after\ prev\ (8\ bytes)\ and\ data\ (4\ bytes).\ Its\ offset\ is\ 12\ bytes\ (8\ +\ 4).$

$$say, newNode \rightarrow next = nullptr;$$

$$\rightarrow baseaddress + offset = 0xff04 + 12\ bytes = 0xff10\begin{bmatrix} 04 + 12 = 16 \\ And\ integer\ 16\ \approx hex\ 10 \end{bmatrix}.$$

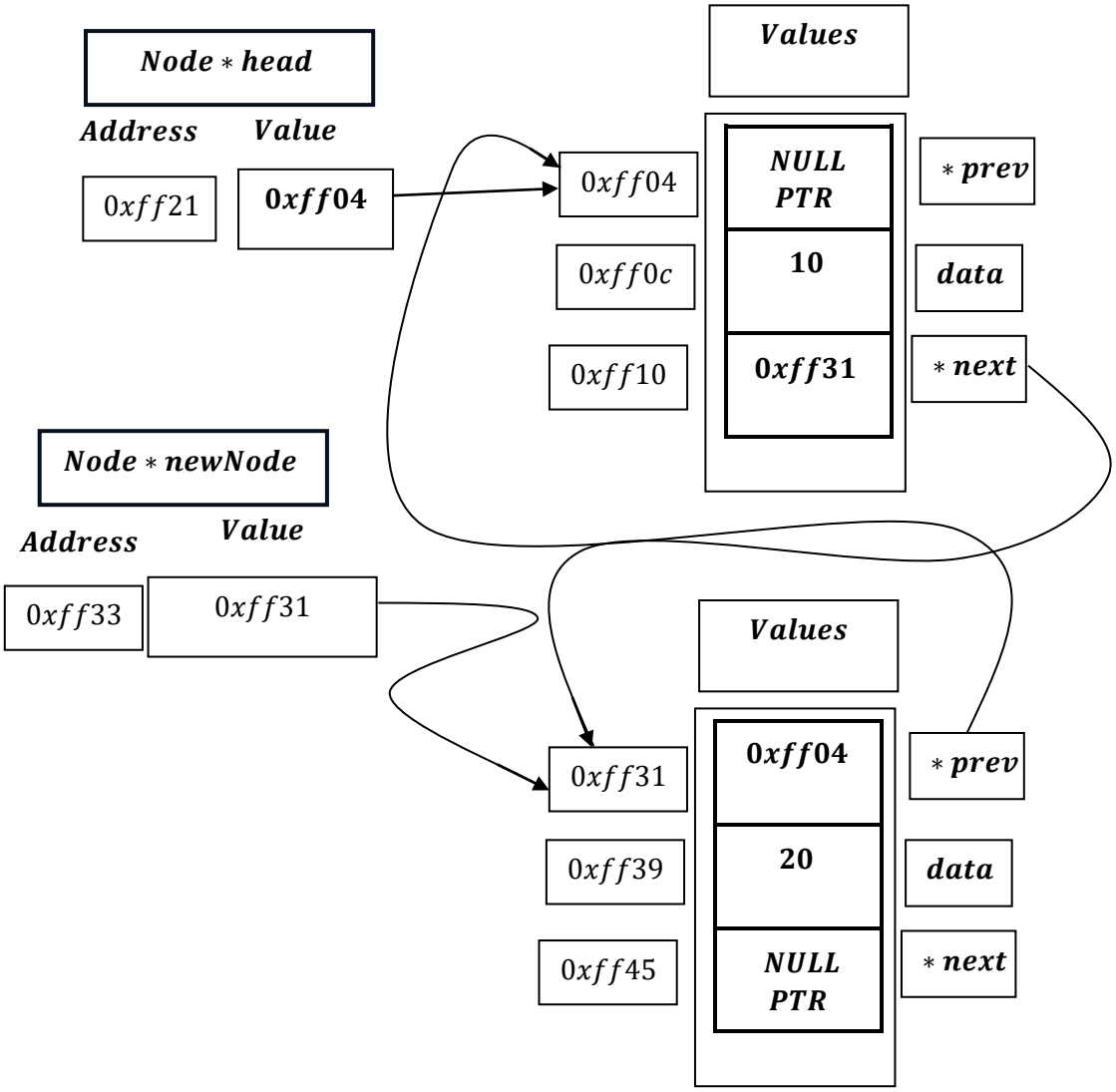
$\rightarrow$ ***And then write head pointer's value to address*** $0xff10$.

When it sees newNode->next = nullptr;, it thinks:

1. *"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)*
2. *"Look up the definition of the Node struct. The next member comes after prev pointer and integer data, which is (8+4) bytes long. So, the offset for next is (8+4)=12."*
3. *"Calculate the final address: base address + offset => 0xff04 + 12 => 0xff10."*
4. *" Write the value `nullptr` or NULL basically (whose output is 0)[pointer points to nothing] to the memory location starting at 0xff10."*

But this doesn't means , value of newNode pointer gets changed and it starts to point to address of next pointer variable, it continue to point to prev pointer .

Now, accessing from prev pointer from newNode pointer:



***newNode → prev → prev*; *Now there is two path*:**

→ ***newNode → prev*: $0xff31 + 0 = 0xff04$ *and new → prev → prev*: $(0xff04) \rightarrow$ *prev*: $0xff04 + 0 = 0xff04$.**

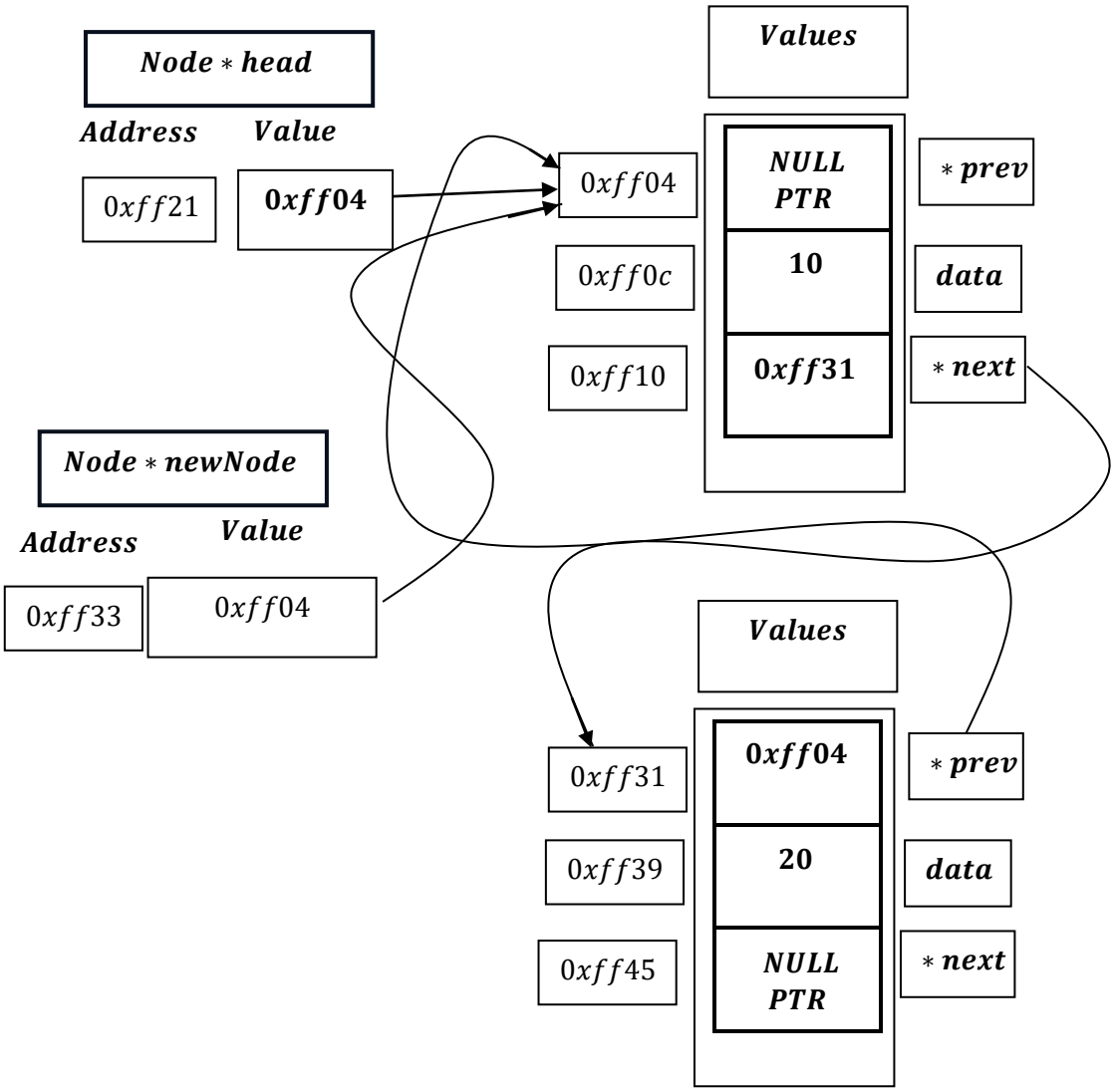
***newNode → prev → data*; *Now there is two path*:**

→ ***newNode → prev*: $0xff31 + 0 = 0xff04$ *and new → prev → data*: $(0xff04) \rightarrow$ *data*: $0xff04 + 8 = 0xff0c$.**

***newNode → prev → next*; *Now there is two path*:**

→ ***newNode → prev*: $0xff31 + 0 = 0xff04$ *and new → prev → next*: $(0xff04) \rightarrow$ *next*: $0xff04 + 12 = 0xff10$.**

***Again*,**



$newNode \rightarrow next \rightarrow prev$; Now there is two path:

$\rightarrow newNode \rightarrow next: 0xff04 + 12 = 0xff31$ and $new \rightarrow next \rightarrow prev: (0xff31) \rightarrow prev: 0xff31 + 0 = 0xff31$.

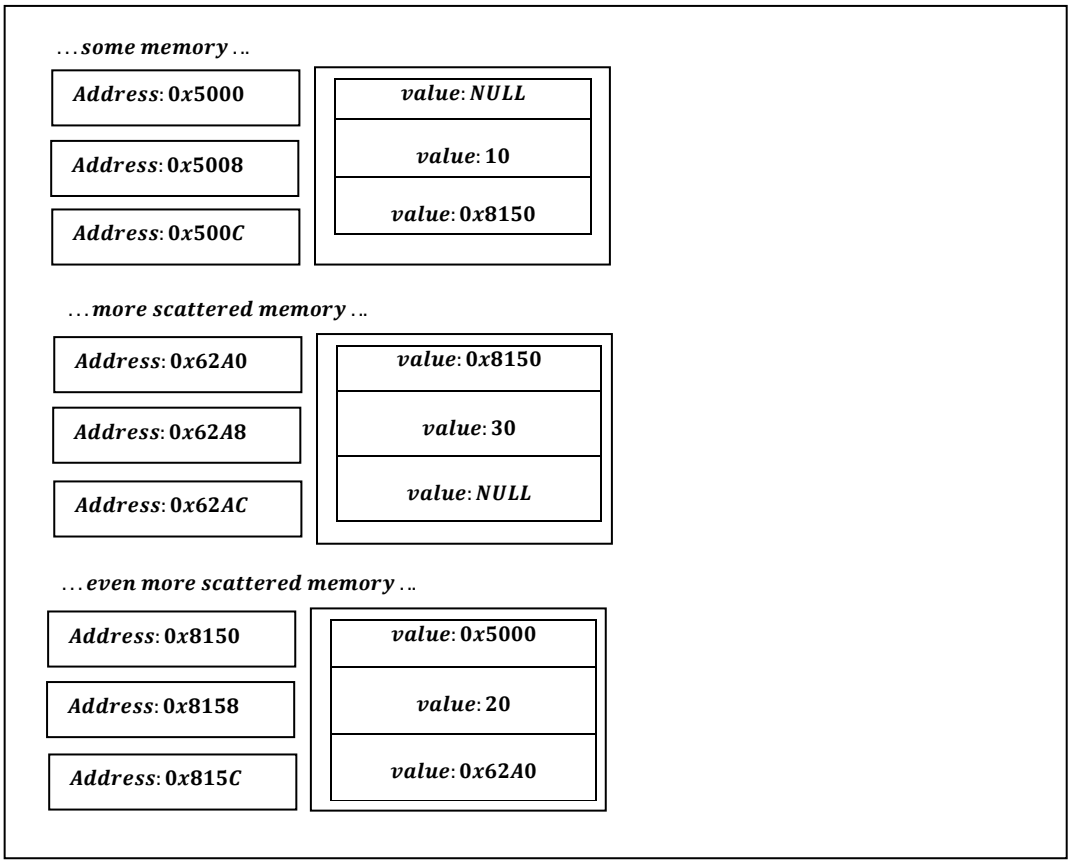
$newNode \rightarrow next \rightarrow data$; Now there is two path:

$\rightarrow newNode \rightarrow next: 0xff04 + 12 = 0xff31$ and $new \rightarrow next \rightarrow data: (0xff31) \rightarrow data: 0xff31 + 8 = 0xff39$.

$newNode \rightarrow next \rightarrow next$; Now there is two path:

$\rightarrow newNode \rightarrow next: 0xff04 + 12 = 0xff31$ and $new \rightarrow next \rightarrow next: (0xff31) \rightarrow next: 0xff31 + 12 = 0xff45$.

How does Heap memory looks like : –



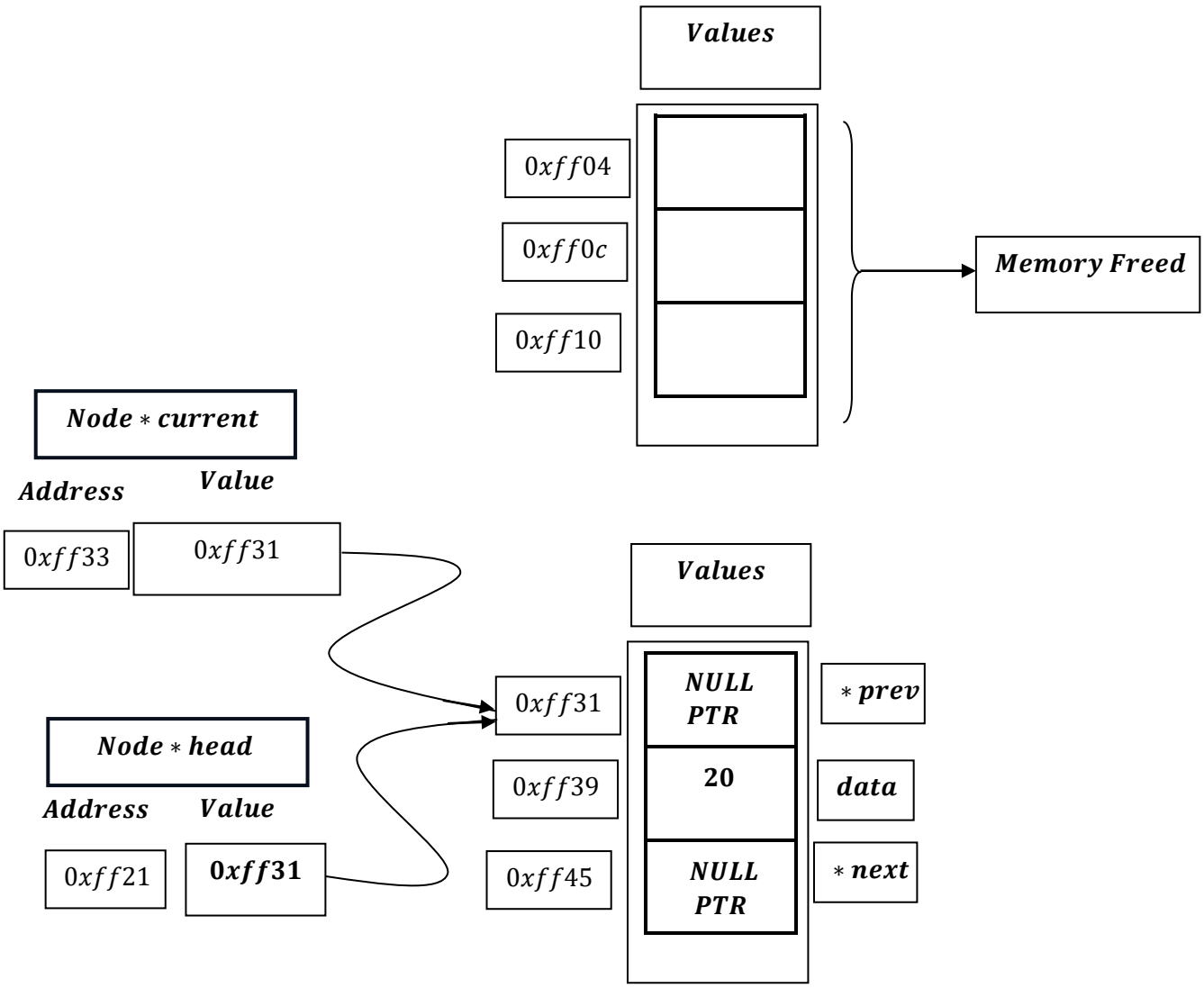
Heap Memory showing how Linked List is formed.

Freeing up the Memory

```
        free(*current);
```

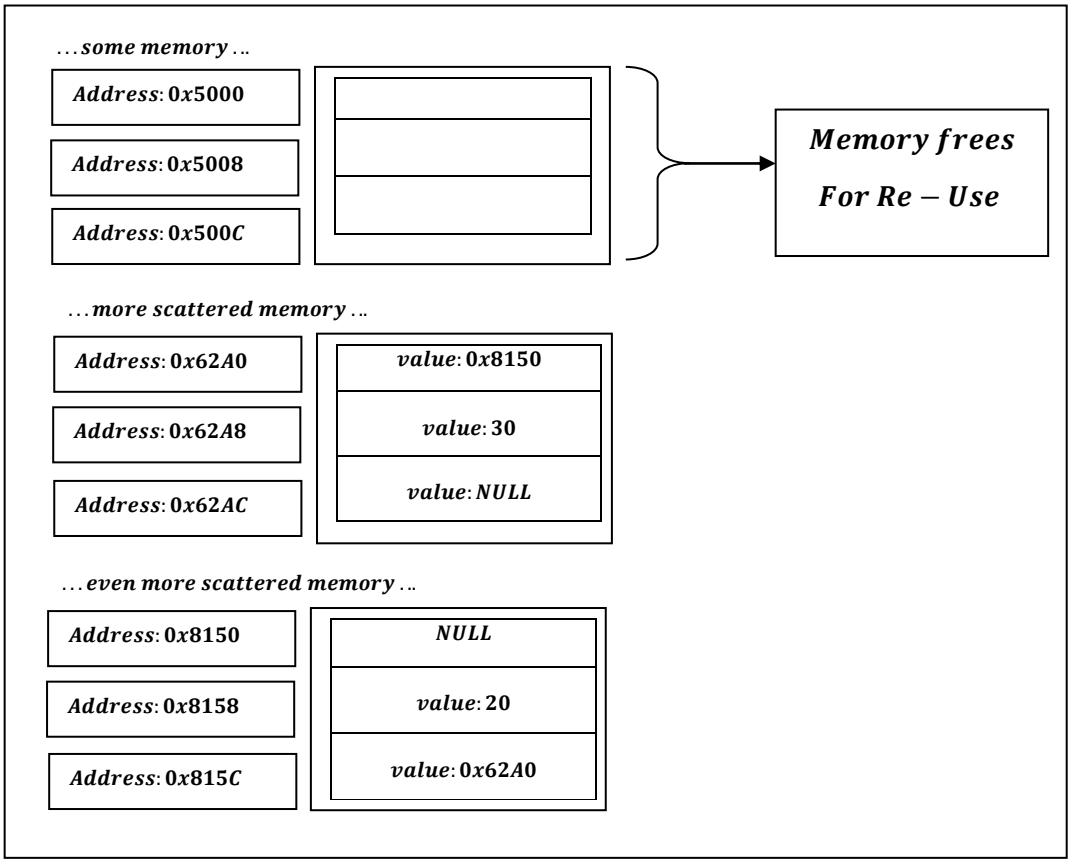
The free(current) command frees the entire memory block allocated for that node, which includes the space for both prev pointer, integer data value and the next pointer.

free () does not know or care that the memory block is being used for a struct with different members. It only knows the starting address (current) and the size of the block that was originally allocated by malloc. It returns that entire chunk of memory to the operating system's heap for reuse.



A simple demonstration of deletion from first node.

***How heap memory looks like*?**


...some memory...
Address: 0x5000
Address: 0x5008
Address: 0x500C
Memory frees
For Re – Use
...more scattered memory...
Address: 0x62A0
Address: 0x62A8
Address: 0x62AC
value: 0x8150
value: 30
value: NULL
...even more scattered memory...
Address: 0x8150
Address: 0x8158
Address: 0x815C
NULL
value: 20
value: 0x62A0